환경 검증과 첫 호출 (실습)

1-1. 환경 점검 스크립트 실행 (실습)
1-2. 자격 증명 설정 (실습)
1-3. 첫 LLM 호출 (실습)
1-4. 실습 레포 준비 (실습)

1-1. 환경 점검 스크립트 실행 (실습)

> 가상환경과 패키지 상태를 코드로 확인해봅니다.

파이썬은 가상환경(venv)과 pip로 의존성을 관리합니다.

둘 다 파이썬을 설치하면 같이 따라오는 기본 도구라 따로 설치할 것은 없습니다.

pip는 파이썬의 패키지 관리자입니다. Java의 Maven과 Gradle이 하는 일을 합니다.



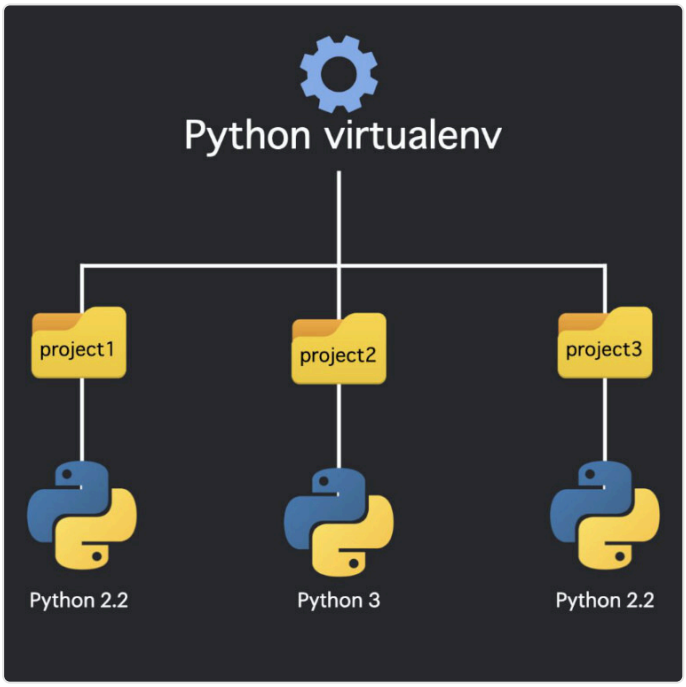
`pip install 이름`을 치면 PyPI(파이썬 공식 패키지 저장소 → Maven Central에 해당)에서 해당 패키지와 그것이 의존하는 패키지를 함께 내려받아 설치합니다.

차이라면 `pom.xml` 처럼 선언 파일이 반드시 있어야 하는 것은 아니고 명령으로 하나씩 설치하는 방식이 기본입니다(목록을 파일로 관리하려면 `requirements.txt` 를 보통 씁니다).

requirements.txt — Edited

```
contourpy==1.0.6
cycler==0.11.0
fonttools==4.38.0
joblib==1.2.0
kiwisolver==1.4.4
matplotlib==3.6.2
numpy==1.23.5
packaging==21.3
pandas==1.5.2
patsy==0.5.3
Pillow==9.3.0
plotly==5.11.0
pyparsing==3.0.9
python-dateutil==2.8.2
pytz==2022.6
scikit-learn==1.1.3
scipy==1.9.3
seaborn==0.12.1
six==1.16.0
statsmodels==0.13.5
tenacity==8.1.0
threadpoolctl==3.1.0
```

가상환경(venv)은 프로젝트마다 독립된 패키지 설치 공간을 만드는 장치입니다. Java에서 프로젝트별 classpath를 분리하는 것과 목적이 같습니다. 가상환경 없이 설치하면 모든 패키지가 시스템 파이썬 한 곳에 쌓여서 다른 프로젝트가 요구하는 버전과 충돌하는 상황을 맞이할 수도 있습니다.

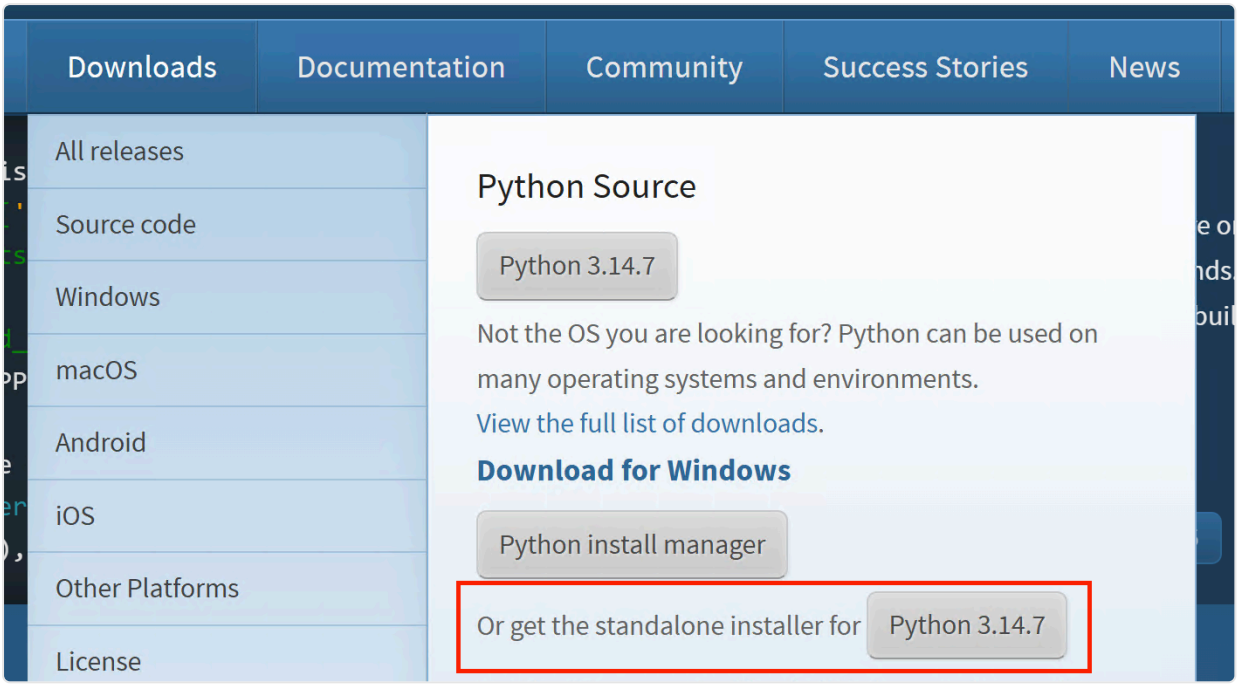


시작 전에 설치할 것

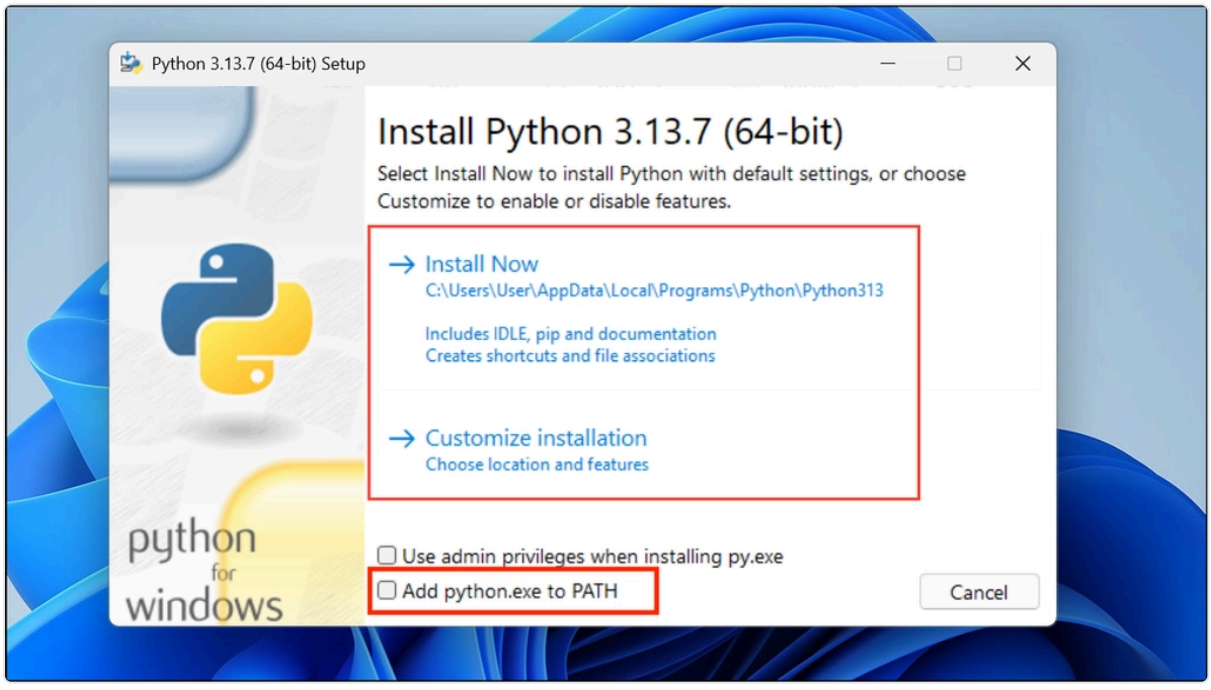
파이썬과 Git을 설치합니다. 이미 깔려 있다면 버전만 확인하고 넘어갑니다.

1) 파이썬 최신버전

- python.org/downloads/windows에서 Windows installer (64-bit)를 내려받습니다.



- 설치 첫 화면 맨 아래의 **Add python.exe to PATH를 반드시 체크**하고 Install Now를 누릅니다. 이걸 놓치면 터미널이 `python` 명령을 못 찾습니다.



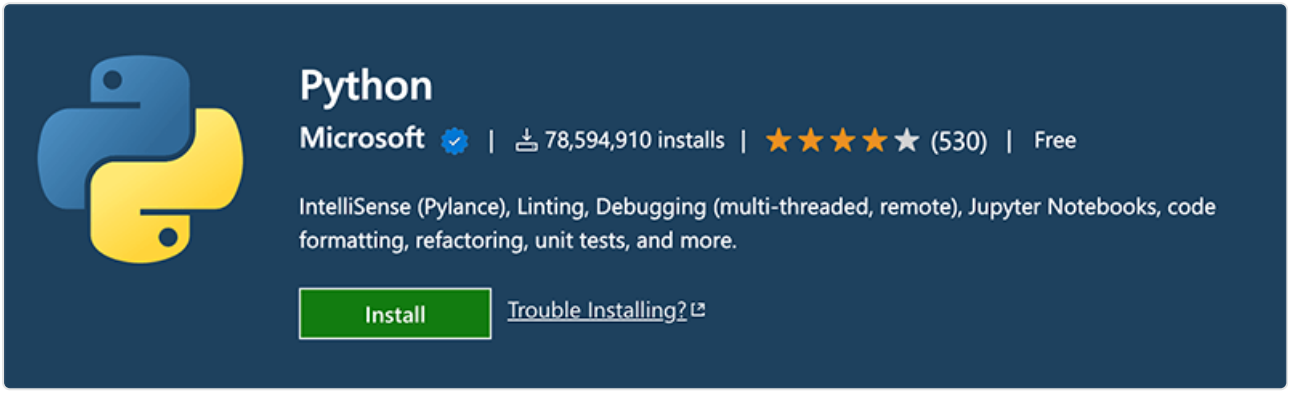
- Microsoft Store에서 받는 파이썬은 설치 경로가 특이하여 가상환경 경로 문제가 생기기 쉬우므로 python.org 홈페이지에서 설치하는 것을 권장드립니다.

2) Git

- git-scm.com/download/win에서 내려받아 옵션은 전부 기본값 그대로 설치 진행합니다.

3) IDE

- VS Code를 강의에서 기본으로 쓰나 다른 IDE 사용하셔도 크게 문제 없습니다. (Antigravity 등)
- https://code.visualstudio.com/
- 혹시라도 VS Code, Antigravity 사용 시 Python 확장 설치 시 가상환경 선택 기능이 있어 편합니다.



설치가 끝나면 **PowerShell 창을 새로 열어** 두 줄로 확인합니다.

```
python --version    # Python 3.11.x 이상이 나와야 합니다
git --version       # git version 2.x
```

따라하기

1. 바탕화면으로 이동, 터미널에서 폴더 두 개를 만듭니다.
2. 바깥쪽 `sds-ax-practice` 는 **7일차까지 쓸 실습 레포의 루트 폴더**이고 그 안의 `day1_practice` 가 오늘 실습 폴더입니다.

3. 내일부터 루트 폴더 아래에 `day2_practice`, `day3_practice`가 나란히 생성되게 됩니다.
   오늘 실습 파일은 모두 `day1_practice` 폴더에 저장합니다.

```
cd Desktop
mkdir sds-ax-practice
cd sds-ax-practice

python -m venv .venv
.venv\Scripts\activate
# 혹시 권한 에러가 발생할 경우 Set-ExecutionPolicy -Scope Process -ExecutionPolicy
Bypass 실행
```

   다음으로 루트 폴더(`sds-ax-practice`)에 **`requirements.txt`** 파일을 만들고 아래 내용을 그대로 붙여 넣습니다. 7일차까지 쓰는 패키지를 한 파일에 모아 둔 것으로 Java의 `pom.xml`이나 `build.gradle`의 의존성 목록에 해당합니다.

```
# 삼성SDS AX 교육 패키지 리스트

# 공통
boto3
python-dotenv

# LangChain (1일차~)
langchain
langchain-aws
langchain-community
langchain-classic
langchain-text-splitters

# RAG (2일차~)
langchain-chroma
rank-bm25
kiwipiepy

# LangGraph (3일차~)
langgraph
langgraph-checkpoint-sqlite
langgraph-supervisor

# MCP (4일차~)
mcp
fastmcp
langchain-mcp-adapters
```

   저장했으면 한 번에 설치를 진행하고 1일차 폴더로 진입합니다.

```
python -m pip install -r requirements.txt

mkdir day1_practice
cd day1_practice
```

`pip` 대신 `python -m pip` 를 쓰는 것이 안전한 편입니다.
지금 활성화된 가상환경의 파이썬으로 설치한다는 것을 명확하게 명시하는 명령어입니다.

패키지가 제대로 깔렸는지는 바로 아래 3번의 점검 스크립트로 확인합니다.

- `python -m venv .venv`

  루트 폴더에서 실행하므로 `venv` 모듈이 `sds-ax-practice\.venv` 폴더를 만들고 그 안에 이 프로젝트 전용 파이썬, 패키지들을 취합합니다. 폴더 이름을 `.venv` 로 쓰는 것은 관례입니다.

- `.venv\Scripts\activate`

  열려있는 터미널에서 시스템 파이썬 대신 `.venv` 안의 파이썬을 쓰도록 전환합니다. 성공하면 프롬프트 앞에 `(.venv)` 가 붙습니다.
  터미널을 새로 열 때마다 다시 실행해야 합니다.

- `pip install A B C`

  패키지를 활성화된 가상환경에 설치합니다. 공백으로 나열하면 한 번에 여러 개를 설치할 수 있습니다.

4. 한글과 이모지가 깨지지 않게 UTF-8 설정을 한 번 해 둡니다.
   실습 간 모델 답변에 이모지가 섞이는 경우가 많은데 이 설정이 없으면 답변을 출력하는 순간 오류가 납니다.

```
setx PYTHONUTF8 1
setx PYTHONIOENCODING utf-8
```

이 설정은 새로 여는 터미널부터 적용됩니다. 이미 열려 있는 창에는 반영되지 않으므로 창을 완전히 닫았다가 다시 엽니다.

적용됐는지 아래로 확인합니다.

```
python -c "import sys; print(sys.stdout.encoding)"
```

`utf-8` 이 나오면 된 것입니다. `cp949` 처럼 다른 값이 나오면 아직 적용되지 않은 것이니 터미널을 완전히 닫았다가 다시 엽니다. 이 설정은 한 번만 해 두면 이후 모든 실습에 적용됩니다.

5. 환경 점검 스크립트를 `day1_practice/check_env.py` 로 저장하고 실행합니다.

```
# check_env.py - 환경 점검 스크립트
import sys
import importlib

print(f"Python 버전: {sys.version}")
assert sys.version_info >= (3, 10), "Python 3.10 이상이 필요합니다."

packages = ["langchain", "langchain_core", "langchain_aws", "dotenv"]
for name in packages:
    try:
        mod = importlib.import_module(name)
        version = getattr(mod, "__version__", "버전 정보 없음")
        print(f"[OK] {name} ({version})")
    except ImportError:
        print(f"[실패] {name} 이 설치되지 않았습니다. pip install 을 확인하세요.")
```

막힐 때

- `UnicodeEncodeError: 'cp949' codec can't encode character` 가 납니다.
  모델 답변에 이모지가 섞였는데 콘솔이 UTF-8이 아닌 경우로, setx PYTHONUTF8 1
  setx PYTHONIOENCODING utf-8 명령어를 순차적으로 실행합니다.

- `python` 명령이 없다고 나옵니다.
  `python --version` 으로 잘 설치되었는지 확인하고 파이썬 설치 시 Add Python to PATH를 체크하지 않았다면 재설치하거나 PATH에 직접 추가합니다.

- 가상환경을 활성화했는데 패키지가 없다고 나옵니다.
  편집기가 다른 인터프리터를 잡고 있는 경우입니다.
  편집기의 파이썬 인터프리터를 `.venv` 안의 파이썬으로 지정합니다.

- `.venv\Scripts\activate` 에서 "이 시스템에서 스크립트를 실행할 수 없으므로" 같은 오류가 납니다.
  PowerShell의 실행 정책(ExecutionPolicy)이 스크립트를 막은 경우입니다.
  현재 터미널 창에만 적용되므로 터미널을 새로 열면 다시 실행해야 합니다.

  ```
  Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
  ```

확인: 네 개 패키지가 모두 `[OK]` 로 출력되면 문제 없이 실습 진행 가능합니다.

1-2. 자격 증명 설정 (실습)

> 교육용 Bedrock 자격 증명을 환경변수로 안전하게 연결해봅니다.

LLM API를 호출하려면 자격 증명이 필요합니다. 파이썬에서는 .env 파일, 즉, 환경변수로 관리하는 것이 관행입니다.

Java에서 `application.properties`를 git에서 제외하고 환경별로 주입하는 것과 같은 방식입니다.

이번 교육은 Amazon Bedrock을 사용합니다. 그래서 API 키 하나가 아니라 AWS 자격 증명을 씁니다.

필요한 값은 access key, secret key, region 세 가지입니다.

따라하기

1. 자격 증명은 메일로 받습니다. 첨부 파일을 루트 폴더(`sds-ax-practice`)로 옮기고 파일 이름을 `.env`로 바꿉니다.
   받은 파일은 이름이 `sds-ax-c1-12.env` 처럼 앞에 번호가 붙어 있습니다. 앞부분을 지워 `.env` 만 남기면 됩니다. 파일 안의 내용은 아래와 같은 형태입니다.

   `load_dotenv()`가 **실행한 파일의 폴더부터 상위로 올라가며 `.env`를 찾기 때문에** 루트에 하나만 두면 `day1_practice` 든 `day7_practice` 든 그대로 읽혀서 날마다 자격 증명 파일을 복사하고 다니지 않아도 되어 편해집니다.

   ```
   # .env 파일 내용
   AWS_ACCESS_KEY_ID=AKIA...
   AWS_SECRET_ACCESS_KEY=...
   AWS_DEFAULT_REGION=us-east-1
   ```

2. `.env` 파일이 git에 올라가지 않게 `.gitignore`를 먼저 만듭니다. 이 파일은 `day1_practice`가 아니라 한 단계 위인 **루트 폴더(`sds-ax-practice/`)** 에 만듭니다.

   ```
   # .gitignore
   .venv/
   .env
   __pycache__/
   ```

3. 아래 코드를 `day1_practice/check_credentials.py` 로 저장하고 실행해 자격 증명이 제대로 읽히는지 확인합니다.

```
# check_credentials.py - 자격 증명 로드 확인
import os
from dotenv import load_dotenv

load_dotenv()  # .env 파일의 내용을 환경변수로 읽어 들입니다

key = os.getenv("AWS_ACCESS_KEY_ID")
if key:
    # 키 전체를 출력하면 안 되므로 앞 4자리만 확인합니다
    print(f"[OK] AWS_ACCESS_KEY_ID 로드됨: {key[:4]}...")
else:
    print("[실패] AWS_ACCESS_KEY_ID 가 없습니다. .env 파일 위치와 내용을 확인하세요.")
```

`load_dotenv()` 는 `.env` 파일을 읽어 프로세스의 환경변수로 올려 줍니다. 그 뒤 `os.getenv("키")` 로 꺼내 쓰는데, 값이 없으면 예외가 아니라 `None` 이 돌아옵니다(Java의 `System.getenv` 와 같습니다). `key[:4]` 는 문자열 앞 4글자만 잘라내는 파이썬 슬라이싱 문법입니다.

막힐 때

- `load_dotenv()` 를 호출했는데 값이 `None` 입니다.
  `.env` 파일이 스크립트 실행 위치에 없는 경우가 대부분입니다. `load_dotenv(find_dotenv())` 를 쓰면 상위 폴더까지 자동으로 찾아 올라갑니다.
- `.env` 에 값을 적을 때 따옴표나 공백 문제
  `KEY=값` 형태로 등호 양옆을 붙여 적는 것이 안전합니다.
- `.env` 를 만들었는데 자격 증명을 못 찾는다고 나옵니다.
  Windows 메모장에서 "UTF-8 (BOM)"으로 저장하면 첫 줄의 키 이름 앞에 보이지 않는 문자가 붙어 값을 읽지 못합니다. 저장할 때 인코딩을 "UTF-8"로 고르거나 VS Code에서 만들어주세요.
- 파일 이름을 `.env` 로 바꿨는데 값을 못 읽습니다.
  Windows 탐색기가 확장자를 숨기고 있으면 실제 이름이 `.env.txt` 가 됩니다. 탐색기 보기 탭에서 파일 확장명 표시를 켜고 이름을 다시 확인합니다. 이름은 `.env` 하나여야 합니다.
- 탐색기에서 이름을 `.env` 로 바꿀 수 없다고 나옵니다.
  점으로 시작하는 이름을 거부하는 경우입니다. 이름 끝에 점을 하나 더 붙여 `.env.` 로 입력하면 마지막 점이 자동으로 지워지면서 `.env` 가 됩니다. 또는 그 폴더에서 명령창을 열어 `ren sds-ax-c1-12.env .env` 를 실행합니다.

확인: `[OK] AWS_ACCESS_KEY_ID 로드됨` 이 출력되면 성공입니다.

1-3. 첫 LLM 호출 (실습)

> 요청과 응답 JSON을 그대로 출력해 LLM API의 원래 모양을 눈으로 확인해봅니다.

LLM API 호출의 본질은 메시지 목록을 보내고 텍스트와 메타데이터를 돌려받는 HTTP 요청입니다.

프레임워크를 쓰기 전에 요청과 응답의 원래 모양을 봐 두면 나중에 LangChain이 무엇을 감싸 주는지 정확히 이해할 수 있습니다.

핵심 개념은 하나입니다. LLM은 다음 토큰(모델이 글을 잘라서 처리하는 단위)을 예측하는 함수입니다. 앞 단어들이 주어지면 다음에 올 가장 그럴듯한 토큰을 확률로 뽑고 그 토큰을 입력 뒤에 붙여 다시 예측하는 일을 반복합니다.

따라하기

1. LangChain 없이 AWS SDK(boto3)로 직접 호출해 원본 응답 구조를 봅니다. 아래 코드를 `day1_practice/first_call_raw.py` 로 저장하고 실행합니다.

```
# first_call_raw.py - 원본 요청/응답 JSON 구조 확인
import json
import boto3
from dotenv import load_dotenv

load_dotenv()
client = boto3.client("bedrock-runtime", region_name="us-east-1")

# 요청: modelId + messages 배열이 핵심입니다
response = client.converse(
    modelId="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    messages=[
        {"role": "user", "content": [{"text": "안녕하세요. 자기소개를 한 문장으로 해주
세요."}]}
    ],
)

# 응답 전체를 JSON으로 출력해 구조를 눈으로 확인합니다
print(json.dumps(response, indent=2, ensure_ascii=False, default=str))
```

2. 출력된 JSON에서 세 가지를 찾아 표시해 둡니다.
   - **messages 배열**: 요청의 중심입니다. 대화가 메시지 객체의 리스트로 표현됩니다.
   - **답변 텍스트**: assistant 역할의 메시지로 돌아옵니다.
   - **usage 필드**: 입출력 토큰 수가 옵니다. 과금과 컨텍스트 관리의 근거가 되는 숫자입니다.

```
{
  "output": {
    "message": {
      "role": "assistant",
      "content": [{"text": "안녕하세요, 저는 질문에 답하고 글을 작성하는 AI 어시스턴트입니다."}]
    }
  },
  "stopReason": "end_turn",
  "usage": {"inputTokens": 23, "outputTokens": 26, "totalTokens": 49}
}
```

막힐 때

- `AccessDeniedException` : 자격 증명은 맞지만 그 계정에서 이 모델을 호출할 권한이 없는 경우입니다. region 이름 오타부터 확인하고, 그래도 나면 발급받은 자격 증명에 이 모델 권한이 포함됐는지 문의합니다.
- `ThrottlingException` : 짧은 시간에 호출이 몰려 분당 호출 한도에 걸린 경우입니다. 몇 초 기다렸다 다시 실행합니다.

확인: 응답 JSON에서 답변 텍스트와 `usage` 의 토큰 수를 확인하였으면 성공적으로 진행한 것입니다.

1-4. 실습 레포 준비 (실습)

> 매일 도전 미션 제출 통로인 본인 GitHub 레포를 만들고 첫 push까지 확인해봅니다.

이 과정에서는 매일 도전 미션 완성 코드를 본인 GitHub 레포에 push해 제출합니다.

매일 제출이 수료 필수 조건이므로 첫날 레포 생성과 첫 push까지 확인해 둡니다.

따라하기

1. GitHub에서 빈 레포(예: `sds-ax-practice`)를 만듭니다.

2. 로컬 폴더를 레포에 연결하고 push합니다.

```
cd ..
git init
git add .
git commit -m "1일차: 환경 검증과 첫 호출"

git remote add origin https://github.com/<본인계정>/sds-ax-practice.git
git branch -M main
git push -u origin main
```

레포는 7일차까지 계속해서 사용할 예정입니다.
